

TECHNICAL DOCUMENTATION

Full-Stack Book Management System

Backend: NestJS + PostgreSQL + Prisma

Frontend: React + Tailwind CSS + Axios

ZOMAC DIGITAL

Applied Software Engineering

May 2026

Table of Contents

- 1. Project Overview**
- 2. Technologies Used**
- 3. System Architecture**
- 4. Database Design**
- 5. API Endpoints Documentation**
- 6. Authentication Flow**
- 7. Installation & Setup Instructions**
- 8. Features & Functionality**
- 9. Security Implementation**
- 10. Testing & Deployment**
- 11. Screenshots**
- 12. Challenges & Solutions**
- 13. Conclusion**

1. Project Overview

Objective

Develop a full-stack web application that enables users to manage their personal book collections. The system provides a complete REST API backend with JWT authentication and a responsive React frontend for comprehensive book management functionality.

Problem Statement

Users lack a simple, professional tool to organize and manage their personal book collections. Existing solutions are either too complex or lack essential features like user authentication, persistent storage, and responsive design.

Solution Overview

A full-stack web application with:

- Secure user authentication using JWT tokens
- PostgreSQL database for persistent storage
- RESTful API with complete CRUD operations
- Modern React frontend with responsive design
- Professional API documentation with Swagger

Key Features

- User Registration & Login with secure password hashing
- Book Management (Create, Read, Update, Delete)
- Protected Routes (requires authentication)
- User-Specific Data Isolation
- Responsive Mobile-Friendly UI
- Comprehensive API Documentation
- Professional Error Handling & Validation

2. Technologies Used

Backend Technologies

Technology	Version	Purpose
NestJS	18.x	REST API Framework
PostgreSQL	12+	Relational Database
Prisma	4.x	ORM & Database Manager
JWT	Latest	Authentication Tokens
Bcrypt	5.x	Password Hashing
Swagger	3.x	API Documentation

Frontend Technologies

Technology	Version	Purpose
React	5.5	UI Framework
React Router	5.5	Client-side Routing
Axios	5.5	HTTP Client
Tailwind CSS	3.x	Utility-first Styling
Vite	5.x	Build Tool

Development Tools

Node.js 18+ - JavaScript runtime

npm - Package manager

Git - Version control

Postman - API testing

3. System Architecture

Architecture Overview

The application follows a three-tier layered architecture:

Presentation Layer (React)

- UI components, pages, and routing. Communicates with API via HTTP/HTTPS.

Application Layer (NestJS)

- Business logic, validation, authentication, and authorization. Processes requests and returns responses.

Data Layer (PostgreSQL + Prisma)

- Database storage and retrieval. Prisma ORM provides type-safe database access.

Request-Response Flow

- User submits form in React component
- Component calls API service method
- API service makes HTTP request with JWT token
- NestJS controller receives request
- Service performs business logic and validation
- Database operation (create/read/update/delete)
- Response sent back to frontend
- React component updates state and UI

Authentication Flow

- User registers: password hashed with bcrypt, user created in database
- User logs in: password verified, JWT token generated
- Token stored in browser localStorage
- Token sent in Authorization header for protected requests
- Server validates token with secret key
- User data attached to request if token valid
- Response returned or 401 error if token invalid

4. Database Design

Entity Relationship Diagram

Two main entities with one-to-many relationship:

User (1) —→ (Many) Books

User Table

Column	Type	Constraint	Description
id	Int	PK	Primary key, auto-incremented
email	String	UNIQUE	Email address, no duplicates
username	String	UNIQUE	Display name, no duplicates
password	String	NOT NULL	Bcrypt hashed password
createdAt	DateTime	DEFAULT	Account creation timestamp
updatedAt	DateTime	AUTO	Last update timestamp

Book Table

Column	Type	Constraint	Description
id	Int	PK	Primary key
title	String	NOT NULL	Book title
author	String	NOT NULL	Author name
isbn	String	UNIQUE	ISBN identifier

publishedYear	Int	NOT NULL	Publication year
description	String	OPTIONAL	Book description
userId	Int	FK	Foreign key to User
createdAt	DateTime	DEFAULT	Creation timestamp
updatedAt	DateTime	AUTO	Last update timestamp

5. API Endpoints Documentation

Base URL

http://localhost:3000

Authentication Endpoints

POST /auth/register

Register a new user account

Request Body:

Field	Type	Example
email	string	user@example.com
username	string	john_doe
password	string (min 6)	password123

Response (201 Created):

```
{ "message": "User registered successfully", "userId": 1 }
```

POST /auth/login

Login user and get JWT token

Request Body:

Book Management System

```
{ "email": "user@example.com", "password": "password123" }
```

Response (200 OK):

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "id": 1, "email": "user@example.com", "username": "john_doe" } }
```

Book Endpoints (Require Authentication)

All book endpoints require Bearer token in Authorization header: Authorization: Bearer YOUR_JWT_TOKEN

POST /books

Create a new book

Request Body:

```
{ "title": "The Pragmatic Programmer", "author": "David Thomas", "isbn": "978-0-13-595705-9", "publishedYear": 1999, "description": "Essential handbook for programmers" }
```

Response (201 Created):

```
{ "id": 1, "title": "The Pragmatic Programmer", "author": "David Thomas", "isbn": "978-0-13-595705-9", "publishedYear": 1999, "description": "Essential handbook for programmers", "userId": 1, "createdAt": "2026-05-23T10:30:00Z", "updatedAt": "2026-05-23T10:30:00Z" }
```

GET /books

Get all books for current user

Response (200 OK):

```
[ { "id": 1, "title": "...", "author": "...", ... }, { "id": 2, "title": "...", ... } ]
```

GET /books/:id

Get single book by ID

Response (200 OK):

```
{ "id": 1, "title": "The Pragmatic Programmer", ... }
```

PATCH /books/:id

Update a book (all fields optional)

Request Body:

```
{ "description": "Updated description" }
```

Response (200 OK):

```
{ "id": 1, "title": "...", "description": "Updated description", ... }
```

DELETE /books/:id

Delete a book

Response (200 OK):

```
{ "id": 1, "title": "...", ... }
```

GET /books/stats/overview

Get book statistics

Response (200 OK):

```
{ "totalBooks": 5 }
```

6. Authentication Flow

Registration Process

- User enters email, username, and password in registration form
- Password validated (minimum 6 characters)
- Email/username checked for duplicates
- Password hashed with bcrypt (10 rounds)
- User record created in database
- Success message returned to frontend

Login Process

- User enters email and password
- User found in database by email
- Provided password compared with stored hash
- If match: JWT token generated with userId and email
- Token expires in 24 hours

Book Management System

- Token returned to frontend
- Token stored in browser localStorage

Protected Request Flow

- Frontend attaches token to Authorization header
- Server extracts token from header
- Token validated using JWT_SECRET
- If valid: payload decoded, userId extracted
- User data attached to request object
- Route handler executes with user context
- If invalid/missing: 401 Unauthorized returned

Logout Process

- Frontend clears token from localStorage
- Token remains valid on server (JWT is stateless)
- Frontend redirects to login page
- Protected routes redirect to login (no token)

Security Features

- Passwords never stored in plain text
- Bcrypt uses random salt per password
- JWT tokens are cryptographically signed
- Tokens cannot be modified without secret
- Each user can only access their own books
- Frontend validates JWT before making requests
- Server validates ownership of resources

7. Installation & Setup Instructions

Prerequisites

Node.js 18 or higher

npm (comes with Node.js)

PostgreSQL 12 or higher

Git

Backend Setup

Clone repository and navigate to backend:

git clone <https://github.com/dynashe-dev/book-management-system>.git

cd book-management-system/backend/book-api

Install dependencies:

npm install

Create environment file (.env):

DATABASE_URL="postgresql://username:password@localhost:5432/book_management_db?schema=public"

JWT_SECRET="your-super-secret-key"

JWT_EXPIRATION="24h"

Create PostgreSQL database:

createdb book_management_db

Run Prisma migrations:

npx prisma migrate dev --name init

Start development server:

npm run start:dev

Backend runs on <http://localhost:3000>

Frontend Setup

Navigate to frontend directory:

cd ../frontend/Book-Management-System

Install dependencies:

npm install

Create environment file (.env.local):

VITE_API_URL=http://localhost:3000

Book Management System

Start development server:

npm run dev

Frontend runs on <http://localhost:5173>

Verify Installation

Backend: Open <http://localhost:3000/api> in browser (Swagger UI)

Frontend: Open <http://localhost:5173> in browser (React app)

Both servers should show no errors in terminal

8. Features & Functionality

Authentication

- User registration with email and username
- Secure login with JWT tokens
- Password hashing with bcrypt
- Protected routes requiring authentication
- Automatic logout on token expiration
- Token stored securely in browser

Book Management

- Create new books with title, author, ISBN, year, description
- View all personal books in list view
- View single book details
- Edit book information (any field)
- Delete books with confirmation
- ISBN uniqueness enforced
- Books sorted by creation date (newest first)

User Interface

- Clean, professional design
- Responsive layout (mobile, tablet, desktop)
- Form validation with error messages
- Loading indicators during API calls
- Error messages for failed operations
- Confirmation dialogs for destructive actions
- Navigation bar with user welcome message

API Features

- REST API with standard HTTP methods
- JSON request/response format
- Comprehensive error handling
- Input validation with DTOs
- Swagger documentation
- CORS enabled
- Proper HTTP status codes

9. Security Implementation

Password Security

- Passwords hashed with bcrypt (10 salt rounds)
- Passwords never logged or exposed
- Password resets not implemented (out of scope)

Authentication Security

- JWT tokens signed with secret key
- Tokens expire after 24 hours
- Tokens cannot be modified without secret
- Tokens validated on every protected request

Data Access Control

- Each user can only access their own books
- Ownership verified on all read/update/delete operations
- 403 Forbidden returned if accessing others' books

Input Validation

- Email format validation
- Minimum password length (6 characters)
- Required fields validation
- ISBN uniqueness check
- Type validation on all inputs

HTTPS Considerations

- In production: Use HTTPS to encrypt token transmission
- In production: Set secure cookie flags
- In production: Enable CORS properly
- In production: Use environment-specific secrets

Recommended Security Practices

- Store JWT_SECRET in environment variables
- Never commit .env files to version control
- Use strong, unique passwords in production
- Implement HTTPS/TLS in production
- Add rate limiting for login attempts
- Implement database backups
- Monitor access logs for suspicious activity

10. Testing & Deployment

Manual Testing Checklist

Authentication

- Register new account
- Login with valid credentials
- Login fails with invalid credentials
- Token stored after login
- Logout clears token
- Cannot access /books without login

Book Management

- Add book with all required fields
- Book appears in list after creation
- Edit book information
- Deletion requires confirmation
- Book removed from list after deletion
- Cannot add book with duplicate ISBN
- Description is optional

API Testing

- Test with Postman
- Verify request headers include token
- Check response status codes
- Validate response data structure
- Test error scenarios (401, 404, 400)

Browser Testing

- Chrome

Performance Considerations

- Database indexes on email and username
- JWT tokens avoid database queries
- Pagination not implemented (nice-to-have for future)
- Images handled client-side only

Deployment Options

Local Development

- npm run start: dev (backend and frontend in separate terminals)

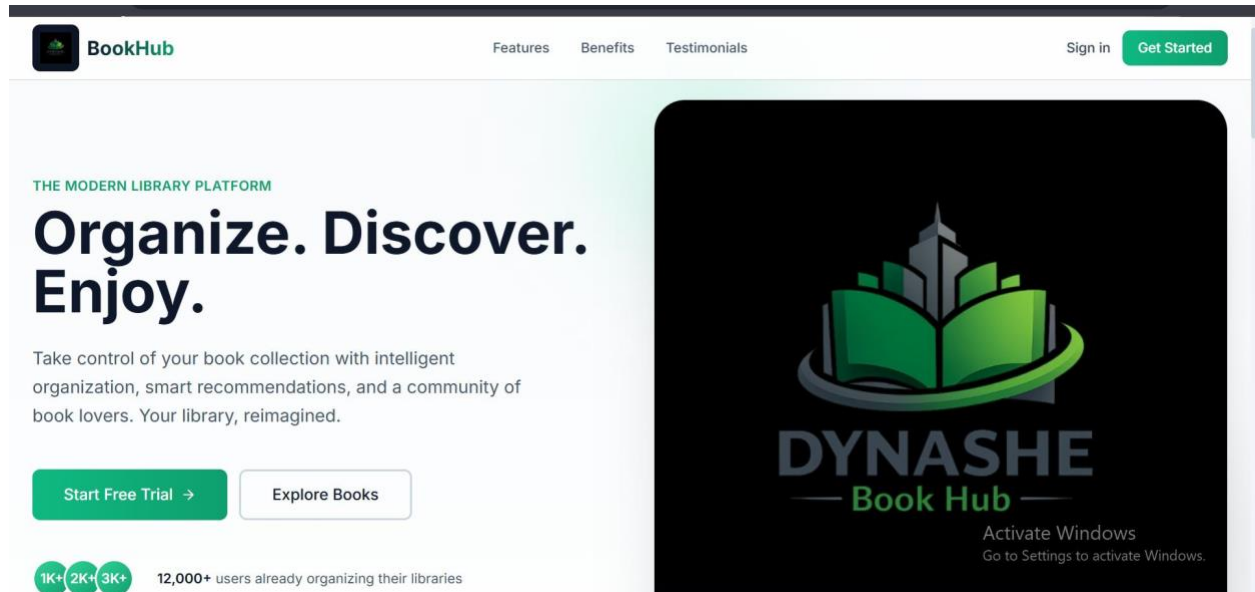
Production Build

- Backend: npm run build && npm start
- Frontend: npm run build (creates dist/ folder)

11. Screenshots

Landing Page

Shows navigation, testimonials, system features with sign in and register options.



Registration Page

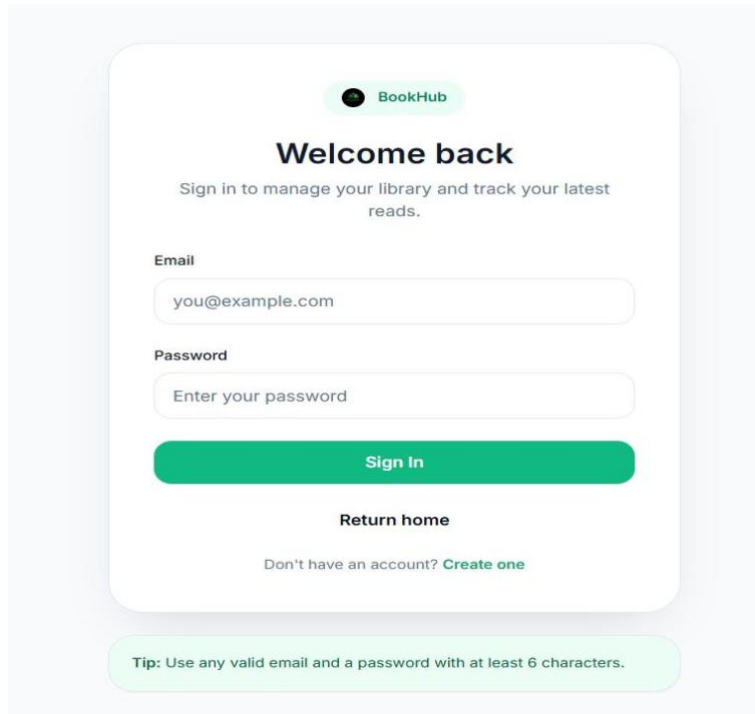
Shows registration form with email, username, password fields. Green color scheme for positive action.

The screenshot shows the registration page of the BookHub application. The page has a clean, modern design with a light gray background. At the top, there is a navigation bar with the BookHub logo. The main heading is "Create an account", followed by a subheading: "Sign up and start managing your books from one polished dashboard." The registration form consists of four input fields: "Username" (placeholder: "Choose a username"), "Email" (placeholder: "you@example.com"), "Password" (placeholder: "Create a password"), and "Confirm Password" (placeholder: "Confirm your password"). Below these fields is a large green button labeled "Create Account". At the bottom of the form, there is a link "Return home" and a link "Already have an account? Sign in". On the right side of the page, there is a partial view of another section with the text "Acti" and "Go to".

Book Management System

Login Page

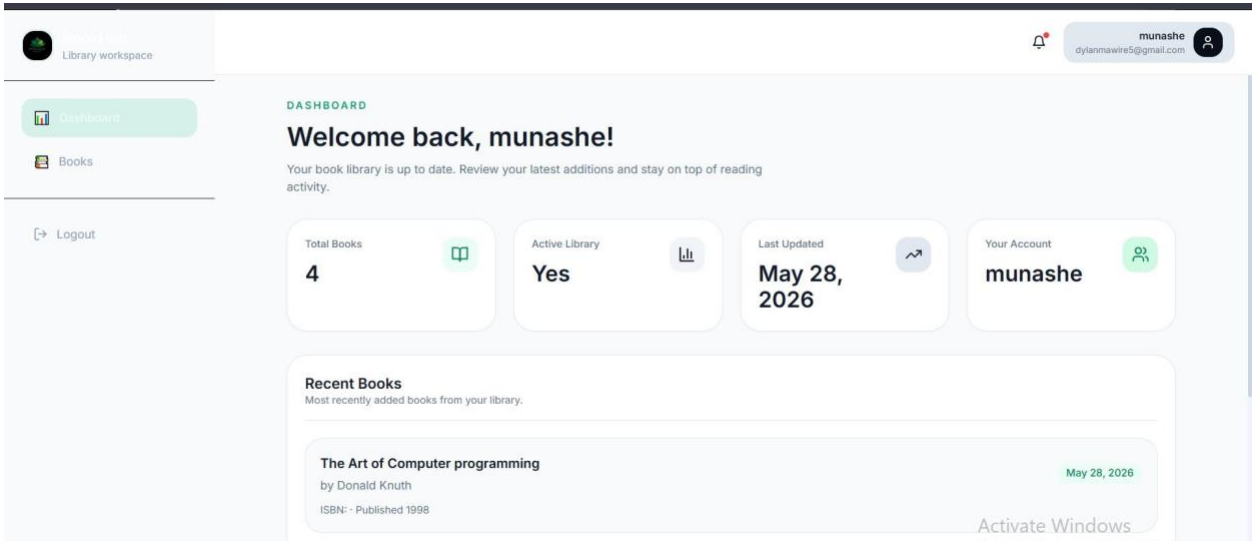
Shows login form with email and password. Blue color scheme. Link to register if no account.



The login form is centered on a light gray background. It features a white rounded rectangle containing the following elements: a 'BookHub' logo at the top, a 'Welcome back' heading, a subheading 'Sign in to manage your library and track your latest reads.', an 'Email' label above a text input field containing 'you@example.com', a 'Password' label above a text input field containing 'Enter your password', a prominent blue 'Sign In' button, a 'Return home' link, and a 'Don't have an account? Create one' link. A green tip box at the bottom states: 'Tip: Use any valid email and a password with at least 6 characters.'

Dashboard

Welcome message with user's name. Statistics showing total books. Quick links to books page.

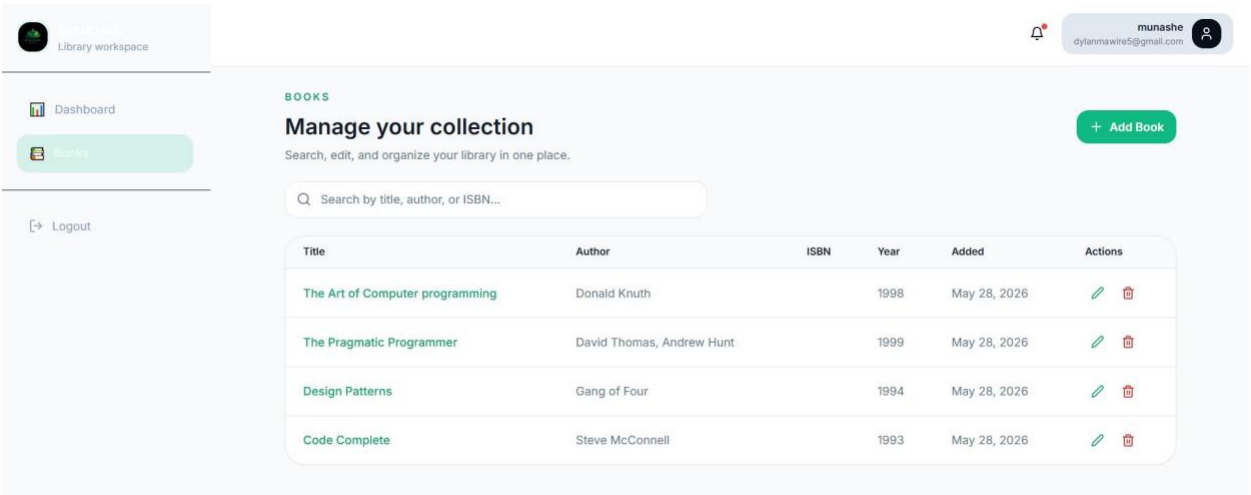


The dashboard interface features a dark sidebar on the left with a 'Library workspace' header, a 'Dashboard' link (highlighted), a 'Books' link, and a 'Logout' button. The main content area has a 'DASHBOARD' header and a 'Welcome back, munashe!' message. Below this is a subheading 'Your book library is up to date. Review your latest additions and stay on top of reading activity.' Four statistics cards are displayed: 'Total Books' (4), 'Active Library' (Yes), 'Last Updated' (May 28, 2026), and 'Your Account' (munashe). A 'Recent Books' section follows, showing 'The Art of Computer programming' by Donald Knuth, published in 1998, with a date tag 'May 28, 2026'. An 'Activate Windows' watermark is visible in the bottom right corner.

Book Management System

Books List Page

Grid of book card. Each card shows title, author, year. Edit and Delete buttons on each card.



Add/Edit Book Form

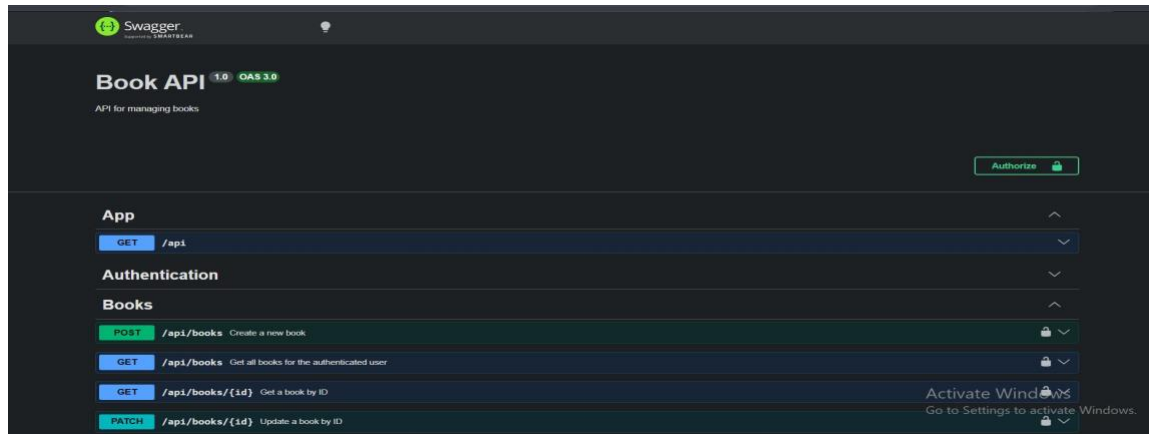
Form with fields for title, author, ISBN, publication year, description. Submit and Cancel buttons. Error messages display above form.



Swagger API Documentation

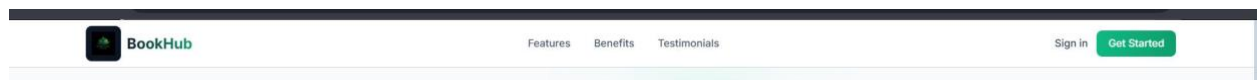
Professional API documentation showing all endpoints, request/response examples, parameters, and authentication requirements.

Book Management System



Navigation Bar

Displays application logo, current user's name, Books link, and Logout button. Responsive design hides elements on mobile.



12. Challenges & Solution

Challenge 1: Ownership Verification

Problem: Users could access other users' books if they knew the ID.

Solution: Added userId check in every book endpoint. Server verifies book belongs to authenticated user before returning/modifying data.

Challenge 2: ISBN Errors

Problem: The system only accepted one isbn code.

Solution: Added @unique to model schema

Challenge 3: Navigation Error

Problem: The system navigated to dashboard after registration

Solution: Created Axios protected routes for secure navigation

Challenge 4: Form Validation

Problem: Invalid data could be sent to API.

Solution: Implemented client-side validation in React forms before submission. Server-side DTOs validate again for security.

Challenge 5: Git commit

Problem: refused to push the project repository due to .git file in backend folder

Solution: ran `rm -rf` command to remove it

13. Conclusion

Project Summary

Successfully developed a full-stack web application demonstrating modern development practices including user authentication, database design, REST API development, and responsive UI design. The application provides a complete solution for managing personal book collections with professional code quality and comprehensive documentation.

Key Achievements

- Implemented secure JWT authentication with bcrypt password hashing
- Designed normalized database schema with proper relationships
- Built production-ready REST API with comprehensive endpoints
- Created responsive React frontend with professional styling
- Generated professional API documentation with Swagger
- Implemented proper error handling and validation throughout

Technical Learnings

- NestJS framework architecture and best practices
- PostgreSQL database design with Prisma ORM
- JWT authentication implementation and security
- React hooks and context API for state management
- Tailwind CSS for responsive design
- RESTful API design principles
- Full-stack development workflow

Potential Enhancements

- User profile management (change password, profile picture)
- Book ratings and reviews
- Search and filter functionality
- Pagination for large book collections
- Export books to PDF
- Social features (share collections)
- Mobile app (React Native)
- Advanced authentication

Book Management System

--- END OF TECHNICAL DOCUMENTATION ---